



Lab Instructions No. 6

Communication and PWM DAC

2026-06-30

Computer Structure Class Spring 2026

Sharif University of Technology

Spring 2026

By Dr. Saeed Bagheri Shouraki

I. Introduction

In this experiment we will explore a basic communication scheme and PWM generation, implemented entirely in MicroPython and simulated with the Wokwi online simulator.

II. Background

II.1. Interfacing with the Emulator via Serial Terminal

You will use the built-in serial terminal of the Wokwi simulator to interact with the μC for this lab. All code must be written in MicroPython and run using only the simulator's built-in parts (the Raspberry Pi Pico, wires, and the Logic Analyzer).

II.2. Simulator Setup Notes

Build each task as a separate Wokwi project (a `diagram.json` together with a `main.py`). Because Wokwi simulates a single microcontroller per project, the exercises that originally required two boards are performed on one Pico using a wired loopback (the transmit pin is connected to the receive pin on the same board). Keep the ADC pins (GP26, GP27, GP28) free for analog use and do not drive them as digital outputs.

III. Tasks

III.1. Command and Control (30 pts)

In this task, you will communicate with the Raspberry Pi Pico using the Wokwi serial terminal. Your MicroPython code should:

1. Accept a command to control the onboard LED brightness using PWM:
 - For example, sending `BRIGHT 73` sets the onboard LED (GP25) brightness to 73%.
2. Respond to the command `TEMP` by reading the onboard temperature sensor (the internal sensor on ADC4) and printing the value to the terminal.

III.2. Software INVERTED UART (30 pts)

In this task, you will implement the UART protocol in software, but with a catch: Line idle should be at low level instead of high level!

Because Wokwi simulates a single board, you will verify your implementation with a self-loopback on one Pico: wire the TX pin to the RX pin on the same board and have your program both transmit and receive.

Steps

1. Software UART Implementation:
 - Implement a software-based UART on pins GP2 (TX) and GP3 (RX)
 - The lines must idle at low-level (standard UART idles at high-level)
 - There is one start bit (high)
 - There are 8 data bits
 - There is no parity bit
 - The baud rate is 9600

- There are two stop bits
2. Loopback and Data Exchange:
 - In the Wokwi diagram, connect GP2 (TX) directly to GP3 (RX)
 - Read an ASCII string from the Wokwi serial terminal, transmit it bit-by-bit on GP2, receive it back on GP3, and print the decoded string to the terminal
 3. Probing:
 - Attach a Wokwi Logic Analyzer to the TX line and capture one full frame
 - Confirm the inverted idle level, the high start bit, the 8 data bits, and the two stop bits in the captured waveform (open the `wokwi-logic.vcd` file in PulseView or `vcd_viewer.html`)

Notes

When moving to real hardware with two boards, remember to transpose the lines (connect TX to RX and vice versa).

III.3. Software SPI (40 pts)

In this task, you will implement the SPI protocol in software.

As before, Wokwi simulates a single board, so you will act as the SPI master and use a loopback: wire MOSI to MISO on the same Pico so that the master reads back the byte it shifts out.

Steps

1. Software SPI Implementation:
 - Use any four pins of your choice (MOSI, MISO, SCLK, CS)
 - Implement the master role (bit-banged SCLK, MOSI as output, MISO as input, CS active-low)
 - You will send 8 data bits at a time (choose a CPOL/CPHA scheme and state it)
2. Loopback and Data Exchange:
 - In the Wokwi diagram, connect MOSI directly to MISO
 - Read an ASCII string from the Wokwi serial terminal, shift each byte out on MOSI, read it back on MISO, and print the received string to the terminal
3. Probing:
 - Attach a Wokwi Logic Analyzer to the SCLK and MOSI pins
 - Run the simulation, stop it, and inspect the captured `wokwi-logic.vcd` waveform (in PulseView or `vcd_viewer.html`) to verify the clock and data timing